

Đại học quốc gia TP.HCM
Trường đại học công nghệ thông tin



Ngành: Khoa học máy tính

Môn học: IT012.P21.KHTN

Lab 4

Sinh viên:

Trần Quang Trường - MSSV: 24521901

Hà Xuân Thiện - MSSV: 24520031

Contents

1	Bài 1: Phân tích ý nghĩa chương trình	2
1.1	Ví dụ 1	2
1.2	Ví dụ 2	2
1.3	Ví dụ 3	3
1.4	Ví dụ 4	3
2	Bài 2: Chương trình Assembly	3
3	Bài 3: Đoạn code điều kiện	7

1 Bài 1: Phân tích ý nghĩa chương trình

1.1 Ví dụ 1

```
1      .data
2 var:      .word    23
3      .text
4 __start:
5      lw      $t0, var1
6      li      $t1, 5
7      sw      $t1, var1
```

Ý nghĩa của chương trình:

- Load giá trị 23 từ var1 vào thanh ghi t0.
- Chèn giá trị 5 của thanh ghi t1 vào var1.

1.2 Ví dụ 2

```
1      .data
2 array1: .space   12
3
4      .text
5 __start:
6      la      $t0, array1
7      li      $t1, 5
8      sw      $t1, ($t0)
9      li      $t1, 13
10     sw      $t1, 4($t0)
11     li      $t1, -7
12     sw      $t1, 8($t0)
```

Ý nghĩa của chương trình:

- Load mảng array1 vào t0, lưu giá trị t1 = 5.
- Lưu giá trị t1 = 5 vào array1[0].
- Load giá trị t1 = 13.
- Lưu giá trị t1 = 13 vào array1[1].

- Load giá trị `t1 = -7`.
- Lưu giá trị `t1 = -7` vào `array1[2]`.

1.3 Ví dụ 3

```

1      li      $v0, 5
2      syscall

```

Ý nghĩa của chương trình:

- Đây là `syscall` với mã 5, nghĩa là sẽ chờ người dùng nhập input từ bàn phím.

1.4 Ví dụ 4

```

1      .data
2      string1: .ascii "Print this.\n"
3
4      .text
5      main:
6          li      $v0, 4
7          la      $a0, string1
8          syscall

```

Ý nghĩa của chương trình:

- Load lệnh 4 (lệnh output) vào thanh ghi `v0`.
- Load xâu `string1` vào thanh ghi `a0`.
- Syscall để in xâu `string1` ra màn hình.

2 Bài 2: Chương trình Assembly

Mô tả: Chương trình thực hiện các thao tác với ba mảng: `array1` (kiểu `.word`), `array2` (kiểu `.byte`), và `array3` (tạo từ `array2`). Chương trình in các mảng, tạo `array3` bằng cách cộng các phần tử đối xứng của `array2`, và cho phép người dùng truy xuất phần tử mảng theo chỉ số.

```

1  .data
2  array1: .word 5, 6, 7, 8, 1, 2, 3, 9, 10, 4
3  size1:  .word 10
4
5  array2: .byte 1, 2, 3, 4, 5, 6,  7, 8, 9, 10, 11, 12,
        13, 14, 15, 16
6  size2:  .word 16
7
8  array3: .space 8
9  size3:  .word 8
10
11 prompt1: .asciiiz "\nNhap mang (1/2/3): "
12 prompt2: .asciiiz "Nhap chi so phan tu: "
13 output:  .asciiiz "Gia tri: "
14 nl:      .asciiiz "\n"
15 space:   .asciiiz " "
16
17 .text
18 .globl main
19
20 main:
21     la    $t0, array1
22     lw    $t1, 0(size1)
23     li    $t2, 0
24
25 print_array1:
26     bge   $t2, $t1, next_array2
27     lw    $a0, 0($t0)
28     li    $v0, 1
29     syscall
30     li    $v0, 4
31     la    $a0, nl
32     syscall
33
34     addi  $t0, $t0, 4
35     addi  $t2, $t2, 1
36     j     print_array1
37
38 next_array2:
39     la    $t0, array2
40     lw    $t1, 0(size2)
41     li    $t2, 0
42
43 print_array2:
44     bge   $t2, $t1, gen_array3

```

```

45     lb    $a0, 0($t0)
46     li    $v0, 1
47     syscall
48     li    $v0, 4
49     la    $a0, n1
50     syscall
51
52     addi $t0, $t0, 1
53     addi $t2, $t2, 1
54     j     print_array2
55
56 gen_array3:
57     li    $t2, 0
58     lw    $t3, 0(size2)
59     addi $t3, $t3, -1
60     la    $t4, array2
61     la    $t5, array3
62
63 gen_loop:
64     lw    $t6, 0(size3)
65     bge   $t2, $t6, input_part
66
67     add   $t7, $t4, $t2
68     lb   $t8, 0($t7)
69
70     sub   $t9, $t3, $t2
71     add   $t9, $t4, $t9
72     lb   $s0, 0($t9)
73
74     add   $s1, $t8, $s0
75     add   $s2, $t5, $t2
76     sb   $s1, 0($s2)
77
78     addi $t2, $t2, 1
79     j     gen_loop
80
81 input_part:
82     li    $v0, 4
83     la    $a0, prompt1
84     syscall
85
86     li    $v0, 5
87     syscall
88     move $t0, $v0
89

```

```

90     li    $v0, 4
91     la    $a0, prompt2
92     syscall
93
94     li    $v0, 5
95     syscall
96     move  $t1, $v0
97
98     li    $t2, 1
99     beq   $t0, $t2, get_array1
100    li    $t2, 2
101    beq   $t0, $t2, get_array2
102    j     get_array3
103
104 get_array1:
105     la    $t3, array1
106     sll   $t1, $t1, 2
107     add   $t4, $t3, $t1
108     lw    $a0, 0($t4)
109     j     print_result
110
111 get_array2:
112     la    $t3, array2
113     add   $t4, $t3, $t1
114     lb    $a0, 0($t4)
115     j     print_result
116
117 get_array3:
118     la    $t3, array3
119     add   $t4, $t3, $t1
120     lb    $a0, 0($t4)
121
122 print_result:
123     li    $v0, 4
124     la    $a0, output
125     syscall
126
127     move  $a0, $t4
128     lb    $a0, 0($a0)
129     li    $v0, 1
130     syscall
131
132     li    $v0, 10
133     syscall

```

3 Bài 3: Đoạn code điều kiện

Mô tả: Đoạn code thực hiện kiểm tra điều kiện $i < j$ và gán giá trị cho $A[i]$ dựa trên kết quả kiểm tra. Nếu $i < j$, thì $A[i] = i$, ngược lại $A[i] = j$.

```
1  .data
2  array: .space 100
3
4  .text
5
6  la $s3, array
7  li $s0, 1
8  li $s1, 0
9
10     li $v0, 5
11     syscall
12     addi $t1, $v0, 0 # t1 = n
13     li $t0, 0 #t0 = i = 0
14 LOOP:
15     slt $t2, $t0, $t1
16     beq $t2, $zero, END #end if t0 >= t1 (i >= n)
17
18
19     li $v0, 5
20     syscall
21     sll $t3, $t0, 2
22     add $t3, $t3, $s3 # Addr of A[i]
23     sw $v0, 0($t3) #A[i] = input
24
25
26     add $t0, $t0, 1 #i++
27     j LOOP
28 END:
29     slt $t2, $s0, $s1
30     beq $t2, $zero, ELSE #if i >= j
31     sll $t3, $s0, 2
32     add $t3, $t3, $s3
33     sw $s0, 0($t3)
34     j ENDIF
```



```
35 ELSE: # i >= j, A[i] = j
36     sll $t3, $s0, 2
37     add $t3, $t3, $s3
38     sw $s1, 0($t3)
39 ENDIF:
40
41 end_if:
```

Listing 2: Đoạn code điều kiện cho Bài 3